DNA Sequence Analysis: GC Content, Motif Detection, and Structural Characterization of *Candida* Species

By Hasna Abbass
30/04/2026

Table of Contents

---

1. Introduction

1.1.Background

DNA sequence analysis is essential in understanding the genetic composition and biological characteristics of organisms. In fungi such as *Candida* species, nucleotide composition (GC and AT content), motif presence, and sequence structure provide insights into gene expression, genome organization, and evolutionary relationships.

GC content is particularly important because it influences DNA stability, gene regulation, and codon usage. Additionally, identifying motifs such as the start codon (ATG) and analyzing sequence structure helps predict protein-coding potential.

1.2. Purpose of the Study

This study aimed to analyze DNA sequences of different *Candida* species using Python. The analysis included:

- Sequence length determination
- Nucleotide composition (A, T, G, C)
- GC and AT content calculation
- Motif (ATG) detection
- RNA transcription and protein translation
- ORF prediction
- Reverse complement generation
- Statistical analysis and visualization

---

2. Objectives

- To determine sequence length and composition
- To calculate GC% and AT%
- To classify sequences based on GC content
- To detect ATG motifs
- To predict coding potential (ORF)
- To visualize sequence characteristics

---

3. Materials and Methods

3.1. Dataset Description

The dataset consists of DNA sequences from different *Candida* species. Each sequence was analyzed for:

- Length
- Nucleotide counts (A, T, G, C)
- GC% and AT%
- Motif presence (ATG)
- Coding potential

3.2. FASTA Sequences

The sequences were stored in a FASTA file (**projectsequences.fasta**). Each entry represents one *Candida* species.

3.3. Tools and Libraries

```
from Bio import SeqIO
from Bio.Seq import Seq
import pandas as pd
import matplotlib.pyplot as plt
```

Role of this code:

This part imports the required Python libraries. SeqIO reads FASTA sequences, Seq is used for sequence manipulation such as translation and reverse complement, pandas organizes results in a table, and matplotlib creates graphs.

3.4.3.4 Data Processing Workflow

1. Read FASTA sequences
2. Compute nucleotide composition
3. Calculate GC% and AT%

4. Perform RNA transcription and protein translation
5. Detect motifs and ORFs
6. Compute N50
7. Generate graphs

---

4. Results

4.1 Data Output

Code used

```
# Name of the FASTA file containing the DNA sequences
fasta_file = "projectsequences.fasta"

# Motif to search inside the DNA sequence
motif = "ATG"

# Empty list to store all computed results for each sequence
data = []

for record in SeqIO.parse(fasta_file, "fasta"):

    seq_id = record.id
    dna = str(record.seq).upper()

    length = len(dna)

    A = dna.count("A")
    T = dna.count("T")
    G = dna.count("G")
    C = dna.count("C")

    gc = (G + C) / length * 100

    if gc < 40:
        gc_class = "Low GC"
    elif gc <= 60:
        gc_class = "Moderate GC"
    else:
        gc_class = "High GC"

    rna = dna.replace("T", "U")

    remainder = len(dna) % 3

    if remainder != 0:
        dna_trimmed = dna[:-remainder]
    else:
```

```
    dna_trimmed = dna

protein = str(Seq(dna_trimmed).translate(to_stop=True))

protein_length = len(protein)

motif_count = dna.count(motif)

reverse_complement = str(Seq(dna).reverse_complement())

if dna.startswith("ATG"):
    start_codon = "Present"
else:
    start_codon = "Absent"

stop_codons = ["TAA", "TAG", "TGA"]
stop_count = 0

for stop in stop_codons:
    stop_count += dna.count(stop)

if dna.startswith("ATG") and stop_count > 0:
    orf_status = "Possible complete ORF"
elif dna.startswith("ATG") and stop_count == 0:
    orf_status = "Start codon only"
else:
    orf_status = "No clear ORF"

at = (A + T) / length * 100

data.append([
    seq_id,
    length,
    A,
    T,
    G,
    C,
    round(gc, 2),
    round(at, 2),
    gc_class,
    motif_count,
    start_codon,
    stop_count,
    orf_status,
    protein_length,
    protein,
    rna,
    reverse_complement
])
```

Role of this code

This code reads each FASTA sequence and calculates the main sequence characteristics, including length, nucleotide counts, GC%, AT%, GC class, RNA sequence, protein sequence, ATG motif count, reverse complement, start codon, stop codon count, and ORF status.

Output

 Complete Results:

	ID	...	Reverse_complement
0	Candida_albicans	...	TTAGCTAACGTTAGCTAGCTAACGTTAGCCTAGCTAGCTAACGTAC...
1	Candida_tropicalis	...	CTAACGTACGCATGCTAACGTTATCTAGCTAACGTTAGCCAAGCTA...
2	Candida_parapsilosis	...	CTAACGCATACTAACGTTAGCTATCTAACGTTAGCCTATCTAGCTA...
3	Candida_glabrata	...	TCAGCTAACGTCAGCTAGCTAACATTAGTCTAGCTATCTAACGTAC...
4	Candida_krusei	...	TAATGTACTCATACTAATGTCAACTAGCTGACATTAGTCTAGCTAT...
5	Candida_auris	...	ACTTATGTCAACTATCTGACATTAGTCTATCTATCTAATGTACTCA...
6	Candida_kefyr	...	TTAGCGCGCCGCGCGGCGCGCCGCGCGGCGCGCCGCGCGGCGCGCC...

[7 rows x 17 columns]

The dataset includes 7 *Candida* species with calculated variables such as GC%, AT%, motif count, and protein sequences.

4.2 N50 Calculation

Code used

```
def calculate_n50(lengths):
    lengths = sorted(lengths, reverse=True)
    total = sum(lengths)
    cumulative = 0

    for length in lengths:
        cumulative += length
        if cumulative >= total / 2:
            return length


n50 = calculate_n50(df["Length"])
print("\n N50 value:", n50)
```

Role of this code

This function calculates the N50 value. N50 is a sequence assembly statistic that indicates the sequence length at which 50% of the total sequence length is reached.

Output

📊 N50 value: 95

Interpretation:

N50 value: **95**. This indicates that at least half of the total sequence length is contained in sequences of length ≥ 95.

4.3. Summary Statistics

Code used

```
print("\n📌 Summary Statistics:")
print("Number of sequences:", len(df))
print("Mean sequence length:", round(df["Length"].mean(), 2))
print("Mean GC%:", round(df["GC%"].mean(), 2))
print("Mean AT%:", round(df["AT%"].mean(), 2))
print("Longest sequence:", df.loc[df["Length"].idxmax(), "ID"])
print("Shortest sequence:", df.loc[df["Length"].idxmin(), "ID"])
```

Role of this code

This code summarizes the general characteristics of the dataset, including the number of sequences, mean length, mean GC%, mean AT%, longest sequence, and shortest sequence.

Output

```
📌 Summary Statistics:
Number of sequences: 7
Mean sequence length: 73.43
Mean GC%: 46.43
Mean AT%: 53.57
Longest sequence: Candida_glabrata
Shortest sequence: Candida_albicans
```

4.4. Saving the Results

Code used

```
df.to_csv("projectsequences_results.csv", index=False)

print("\n✅ Results saved as: projectsequences_results.csv")
```

Role of this code

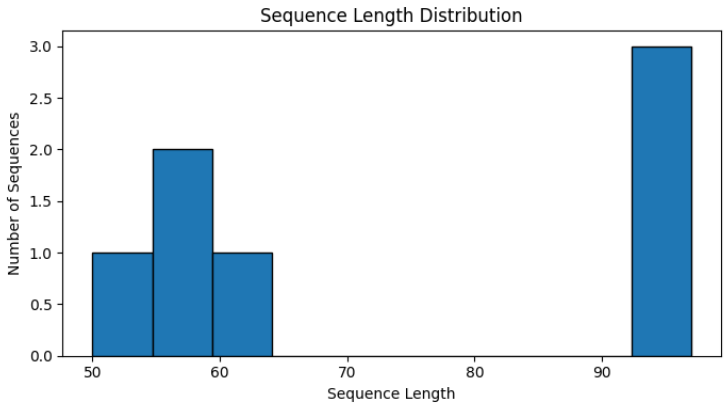
This code saves the final table containing all calculated results into a CSV file. This file can be opened later in Excel or reused for further analysis.

4.5. Graphical Analysis

Figure 1. Sequence Length Distribution

Code used

```
plt.figure(figsize=(7, 4))
plt.hist(df["Length"], bins=10, edgecolor="black")
plt.title("Sequence Length Distribution")
plt.xlabel("Sequence Length")
plt.ylabel("Number of Sequences")
plt.tight_layout()
plt.show()
```



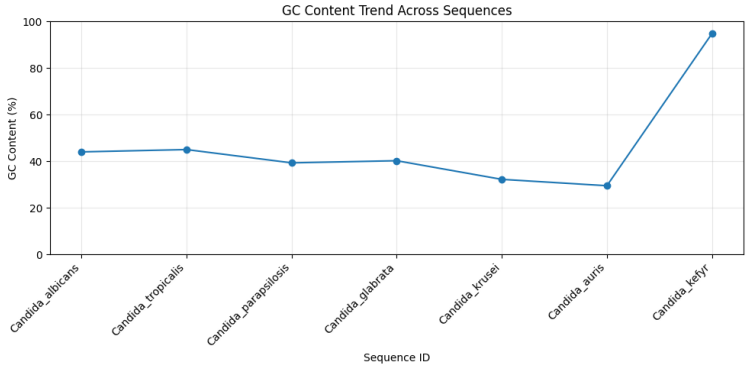
Interpretation:
The histogram shows variation in sequence lengths among *Candida* species. Most sequences are clustered between 50 and 65 base pairs, while a few sequences are longer (~95 bp), indicating heterogeneity in sequence size.

Figure 2. GC Content Trend Across Sequences

Code used

```
plt.figure(figsize=(10, 5))
plt.plot(df["ID"], df["GC%"], marker="o", linestyle="-")
plt.title("GC Content Trend Across Sequences")
plt.xlabel("Sequence ID")
```

```
plt.ylabel("GC Content (%)")
plt.xticks(rotation=45, ha="right")
plt.ylim(0, 100)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

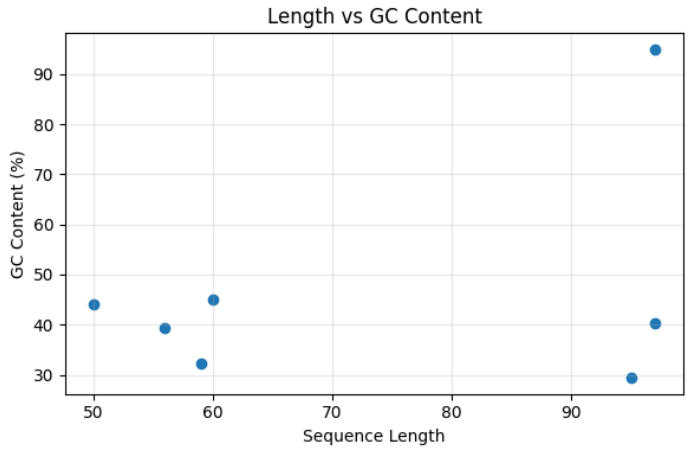


Interpretation:
GC content varies between species. Most *Candida* species show moderate GC levels (~30–45%), while one sequence (*Candida_kefyr*) shows a very high GC content (~95%), indicating strong nucleotide composition differences.

Figure 3. Length vs GC Content

Code used

```
plt.figure(figsize=(6, 4))
plt.scatter(df["Length"], df["GC%"])
plt.title("Length vs GC Content")
plt.xlabel("Sequence Length")
plt.ylabel("GC Content (%)")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



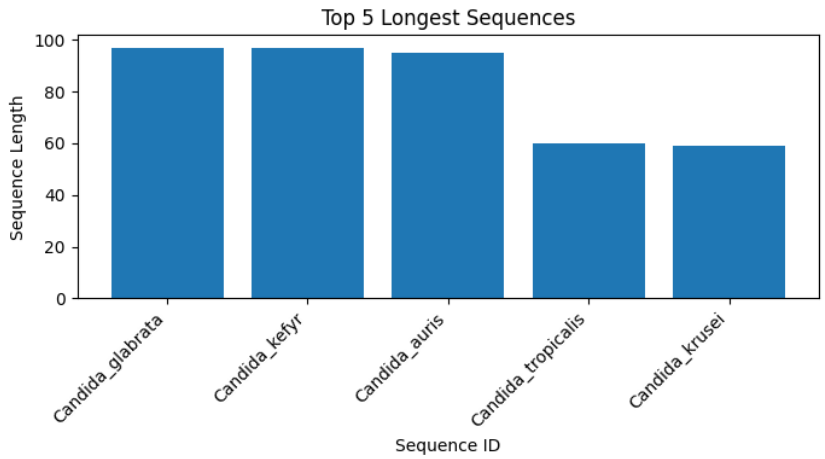
Interpretation:
There is no strong correlation between sequence length and GC content. Both short and long sequences display a wide range of GC percentages, suggesting that GC content is independent of sequence length in this dataset.

Figure 4. Top 5 Longest Sequences

Code used

```
top5 = df.sort_values(by="Length", ascending=False).head(5)

plt.figure(figsize=(7, 4))
plt.bar(top5["ID"], top5["Length"])
plt.title("Top 5 Longest Sequences")
plt.xlabel("Sequence ID")
plt.ylabel("Sequence Length")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



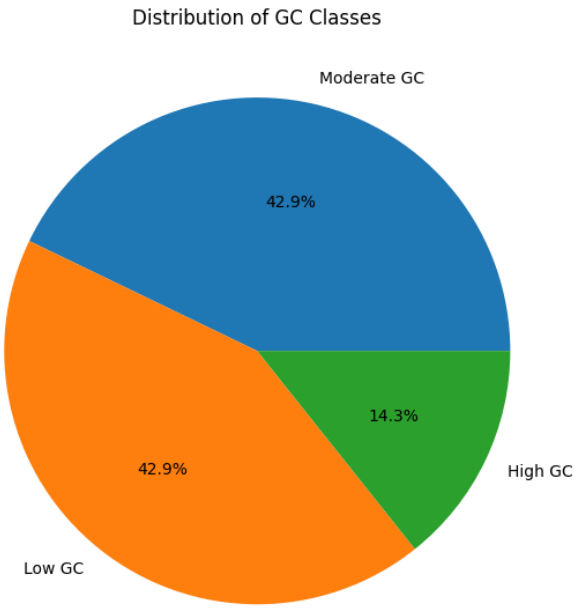
Interpretation:
The longest sequences include *Candida_glabrata*, *Candida_kefyr*, and *Candida_auris*. These sequences contribute significantly to the total sequence length and influence the N50 value.

Figure 5. Distribution of GC Classes

Code used

```
gc_counts = df["GC_class"].value_counts()

plt.figure(figsize=(6, 6))
plt.pie(gc_counts, labels=gc_counts.index, autopct="%1.1f%%")
plt.title("Distribution of GC Classes")
plt.tight_layout()
plt.show()
```



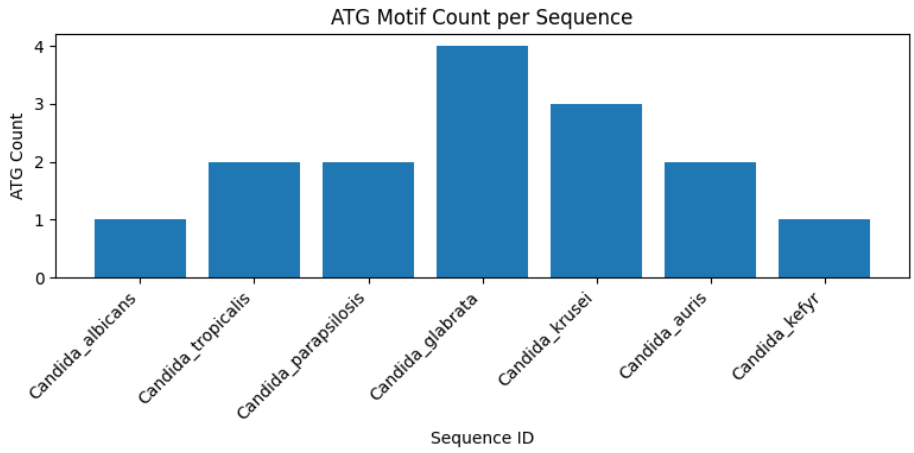
Interpretation:

The dataset is composed mainly of low and moderate GC sequences (~85%), with a smaller proportion of high GC sequences (~14%). This reflects typical fungal genome composition, with some exceptions.

Figure 6. ATG Motif Count per Sequence

Code used

```
plt.figure(figsize=(8, 4))
plt.bar(df["ID"], df["ATG_motif_count"])
plt.title("ATG Motif Count per Sequence")
plt.xlabel("Sequence ID")
plt.ylabel("ATG Count")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



Interpretation:
The number of ATG motifs varies across sequences. *Candida_glabrata* shows the highest count, suggesting a higher potential for translation initiation sites, while other species have fewer motifs.

---

5. General Interpretation

The analysis demonstrates variability in sequence length, nucleotide composition, and motif distribution among *Candida* species.

GC content differences highlight diversity in genomic structure, while motif analysis provides insight into potential coding regions.

The absence of a strong correlation between length and GC content suggests that sequence composition is species-dependent rather than size-dependent.

Overall, the combination of sequence statistics and visualization provides a comprehensive understanding of the structural characteristics of the analyzed DNA sequences.

---

6. Conclusion

This study successfully analyzed DNA sequences of *Candida* species using Python-based tools. The results demonstrated variation in sequence length, GC content, and motif distribution.

The integration of statistical analysis and graphical visualization allowed clear interpretation of sequence characteristics.

This approach highlights the importance of computational tools in molecular biology and provides a foundation for more advanced genomic analysis.